

CHEATSHEET OpenACC for C

This cheatsheet summarizes the essential OpenACC directives and clauses covered in the lecture slides. It is grouped by functionality to help you quickly find the right command for your code.

1. Basic Parallelization Constructs

These directives tell the compiler to offload code to the GPU.

Kernels Construct

- **Syntax:** `#pragma acc kernels [clauses]`

- **Example:**

```
C
#pragma acc kernels
{
    for (int i = 0; i < N; ++i) {
        y[i] = a * x[i] + y[i];
    }
}
```

- **Usage:** Use this when you want the compiler to automatically analyze dependencies and parallelize loops for you¹¹¹. It is "safer" but gives you less manual control.

+1

Parallel Construct

- **Syntax:** `#pragma acc parallel [clauses]`

- **Example:**

```
C
#pragma acc parallel num_gangs(32) num_workers(128)
{
    #pragma acc loop
    for (int i = 0; i < N; ++i) {
        // ...
    }
}
```

- **Usage:** Use this when you want explicit control over parallelism (creating "gangs" and "workers" manually)²²². It executes the structured block redundantly on all gangs unless

you specify a loop inside.
+1

2. Data Management (The data Region)

These clauses control the movement of memory between Host (CPU) and Device (GPU). They are usually attached to `#pragma acc data` or `#pragma acc kernels/parallel`.

General Data Movement

- **Syntax:** `#pragma acc data clause(list)`
- **Clauses:**
 - `copy(list)`: **In & Out**. Copies data Host $\$$ to $\$$ Device at start, and Device $\$$ to $\$$ Host at end³.
 - `copyin(list)`: **In Only**. Copies Host $\$$ to $\$$ Device at start. No copy back⁴.
 - `copyout(list)`: **Out Only**. Allocates on Device. Copies Device $\$$ to $\$$ Host at end⁵.
 - `create(list)`: **Allocation Only**. Allocates scratchpad memory on Device. No copying⁶.
 - `delete(list)`: Deallocates memory on the device (implied at end of region, but can be explicit).

- **Example:**

```
C
// 'A' is input/output, 'B' is input only, 'tmp' is scratchpad
#pragma acc data copy(A[0:N]), copyin(B[0:N]), create(tmp[0:N])
{
    #pragma acc kernels
    for(int i=0; i<N; i++) {
        tmp[i] = B[i] * 2; // Use scratchpad
        A[i] += tmp[i];    // Update output
    }
} // A is copied back here, B and tmp are freed on GPU
```

- **Usage:** Use `copy` for read-write data, `copyin` for read-only lookup tables, `copyout` for final results overwriting previous data, and `create` for temporary intermediate arrays⁷.

Handling Existing Data (present)

- **Clauses:**

- `present(list)`: Assumes data is *already* on the GPU. Errors if not⁸.
- `present_or_copy(list)` (or `pcopy`): Checks if present; if yes, uses it. If no, performs a copy.
- `present_or_create(list)` (or `pcreate`): Checks if present; if yes, uses it. If no, performs a create.
- **Example:**

```
C
void process_data(float* A, int N) {
    // Safe to call from CPU or from inside another GPU region
    #pragma acc data present_or_copy(A[0:N])
    {
        // ... kernels ...
    }
}
```

Interoperability (`deviceptr`)

- **Syntax:** `deviceptr(list)`
- **Example:**

```
C
// ptr is a raw CUDA pointer allocated via cudaMalloc
#pragma acc parallel deviceptr(ptr)
{
    // ... access ptr directly ...
}
```
- **Usage:** Use this when your pointer already points to a GPU address (e.g., from a CUDA library call) and you want OpenACC to use it without translation⁹.

3. Data Synchronization & Updates

Commands to manually move data in the middle of a compute region.

Update Directive

- **Syntax:** `#pragma acc update [clause]`
- **Clauses:**
 - `host(list)`: Copies Device $\$to\$$ Host.
 - `device(list)`: Copies Host $\$to\$$ Device.
- **Example:**

```

C
#pragma acc data copy(A[0:N])
{
    #pragma acc kernels
    { /* modify A on GPU */ }

    // Sync just one element to check convergence on CPU
    #pragma acc update host(A[0:1])

    if(A[0] < tolerance) break;
}

```

- **Usage:** Use to synchronize specific variables (like error flags or debug values) without ending the entire data region¹⁰¹⁰¹⁰¹⁰.
+1

4. Loop Optimization

These clauses give you control over *how* the iterations are distributed across the GPU hardware.

Loop Construct

- **Syntax:** `#pragma acc loop [clauses]`
- **Clauses:**
 - `gang`: Distributes iterations across Streaming Multiprocessors (SMs)¹¹.
 - `worker`: Distributes iterations across threads/warps within an SM¹².
 - `vector`: Uses SIMD lanes (threads) for the innermost operations¹³.
 - `collapse(N)`: Merges N nested loops into one long loop¹⁴.
 - `independent`: Asserts that iterations have no data dependencies¹⁵.
 - `seq`: Forces sequential execution (no parallelism)¹⁶.
 - `reduction(op:var)`: Handles combining results (e.g., sum, max) safely¹⁷.
- **Example (Matrix Multiplication style):**

C

```

#pragma acc parallel loop gang
for (int i = 0; i < N; ++i) {
    #pragma acc loop worker
    for (int j = 0; j < M; ++j) {
        float sum = 0;
        #pragma acc loop vector reduction(+:sum)
        for (int k = 0; k < K; ++k) {
            sum += A[i][k] * B[k][j];
        }
        C[i][j] = sum;
    }
}

```

- **Usage:** Use gang for outer loops, vector for inner loops, and collapse to increase parallelism when loops are small¹⁸¹⁸¹⁸¹⁸.
+1

5. Advanced & Runtime Controls

Cache Directive (Shared Memory)

- **Syntax:** `#pragma acc cache(list)`
- **Example:**

```

C
#pragma acc loop
for (int i = 0; i < N; ++i) {
    // Load 'tile' into fast shared memory
    #pragma acc cache(A[i:tile_size])
    // ... compute ...
}

```

- **Usage:** Explicitly loads a subset of data into "software managed data cache" (GPU Shared Memory) for faster access¹⁹¹⁹¹⁹¹⁹.
+1

Async & Wait

- **Syntax:**
 - `async(id)`: Put on a parallel/kernels directive to run in background²⁰.
 - `#pragma acc wait(id)`: Block CPU until queue id is finished²¹.

- **Example:**

```
C
#pragma acc kernels async(1)
{ /* kernel 1 */ }

// CPU can do other work here...

#pragma acc wait(1) // Wait for kernel 1 before proceeding
```

- **Usage:** Essential for overlapping CPU and GPU work or managing dependencies between two different kernels.

6. Execution Control Clauses

These clauses control *whether* or *how* a parallel region starts. They can be attached to `#pragma acc kernels` or `#pragma acc parallel`.

Conditional Execution (if)

- **Syntax:** `if(condition)`
- **Example:**

```
C
// Only offload to GPU if N is large enough to be worth the overhead
#pragma acc kernels if(N > 1000)
for(int i = 0; i < N; i++) {
    A[i] = B[i] * 2.0f;
}
```

- **Usage:** Prevents small workloads from running on the GPU, where data transfer time would exceed compute time. If false, the code runs on the CPU.

Asynchronous Execution (async)

- **Syntax:** `async(integer_expression)`
- **Example:**

```
C
// Launch kernel on queue #1
#pragma acc parallel loop async(1)
for(int i = 0; i < N; i++) A[i] = i;

// CPU continues immediately here...
// Launch another independent kernel on queue #2
#pragma acc parallel loop async(2)
```

```
for(int i = 0; i < N; i++) B[i] = i;
```

```
// Wait for both to finish before using results
```

```
#pragma acc wait(1, 2)
```

- **Usage:** Allows the CPU to keep working or launch multiple GPU kernels simultaneously (on different streams/queues).

7. Parallel Configuration Clauses (The "Setup")

These clauses are strictly used with the `#pragma acc parallel` construct. They define the **total resources** allocated for the kernel (similar to setting `dim3 grid` and `dim3 block` in CUDA).

`num_gangs`, `num_workers`, `vector_length`

- **Syntax:**
 - `num_gangs(expression)`: Sets the number of blocks (Grids).
 - `num_workers(expression)`: Sets the number of warps per block.
 - `vector_length(expression)`: Sets the number of threads per warp (SIMD width).

- **Example:**

```
C
```

```
// Manually configure grid and block dimensions
```

```
// 32 Gangs (Blocks), 4 Workers (Warps), 32 Vector Length (Threads)
```

```
// Total threads = 32 * 4 * 32
```

```
#pragma acc parallel num_gangs(32) num_workers(4) vector_length(32)
```

```
{
```

```
    // Inside this region, the hardware is configured.
```

```
    // You must now use 'loop' to distribute work onto this hardware.
```

```
    #pragma acc loop gang
```

```
    for(int i = 0; i < N; i++) { ... }
```

```
}
```

- **Usage:** Use these when you need to tune performance for specific hardware limitations or occupancy requirements.

8. Loop Mapping Clauses (The "Distribution")

These clauses are used with `#pragma acc loop` inside a parallel or kernels region. They tell the compiler **how to split the loop iterations** onto the resources you set up.

`gang`, `worker`, `vector`

- **Syntax:**
 - gang: Distribute loop iterations across Gangs (Blocks).
 - gang(num): (Kernels only) Specifies number of gangs *and* distributes.
 - worker: Distribute iterations across Workers (Warps).
 - worker(num): (Kernels only) Specifies number of workers *and* distributes.
 - vector: Distribute iterations across Vector Lanes (Threads).
 - vector(len): (Kernels only) Specifies vector length *and* distributes.
- **Example (Hierarchical Mapping):**

```
C
#pragma acc parallel // Hardware limits auto-selected or set previously
{
    // Outer loop goes to Gangs (Blocks)
    #pragma acc loop gang
    for (int i = 0; i < N; ++i) {

        // Inner loop goes to Vectors (Threads)
        #pragma acc loop vector
        for (int j = 0; j < M; ++j) {
            A[i][j] = val;
        }
    }
}
```

- **Usage:** gang is for coarse-grained parallelism (outer loops). vector is for fine-grained SIMD parallelism (inner loops). worker is rarely used but sits in between.

9. Variable Scoping Clauses

These control how variables are shared or privatized among threads.

private

- **Syntax:** private(list)
- **Example:**

```
C
#pragma acc parallel loop private(tmp)
for(int i = 0; i < N; i++) {
    float tmp = A[i] * 0.5f; // Every thread gets its own 'tmp'
    B[i] = tmp + 1.0f;      // 'tmp' is uninitialized at start of iteration
}
```

- **Usage:** Essential for scalar variables defined *outside* the loop but used *inside* as scratch

space. Prevents race conditions.

firstprivate

- **Syntax:** firstprivate(list)
- **Example:**

```
C
float scale = 5.0f;
#pragma acc parallel loop firstprivate(scale)
for(int i = 0; i < N; i++) {
    scale += 1.0f; // Modifies local copy, starting at 5.0f
    A[i] = B[i] * scale;
}
// Host 'scale' remains 5.0f
```

- **Usage:** Like private, but the variable starts with the value it had on the CPU before the loop began.

10. Interoperability

host_data

- **Syntax:** #pragma acc host_data use_device(list)
- **Example:**

```
C
#pragma acc data copy(A[0:N]) // A is on GPU
{
    // We need the raw GPU pointer to pass to a CUDA library function
    #pragma acc host_data use_device(A)
    {
        // 'A' here is the GPU address, not the CPU address
        cublasSscal(handle, N, &alpha, A, 1);
    }
}
```

- **Usage:** Used *only* when you need to pass an OpenACC-managed array to a function that expects a raw device pointer (like NVIDIA cuBLAS or cuFFT).

11. Detailed Loop Optimization Examples

Here are the specific examples you requested for the optimization clauses.

collapse(N)

- **Definition:** Merges N nested loops into one long loop to increase the total iteration count (parallelism).
- **Example:**

```
C
// N=100, M=100. Without collapse, gang might only get 100 items.
// With collapse(2), it sees 10,000 items.
#pragma acc parallel loop collapse(2)
for (int i = 0; i < 100; i++) {
    for (int j = 0; j < 100; j++) {
        A[i][j] = 0.0f;
    }
}
```

independent

- **Definition:** Overrides compiler analysis to assert that iterations do not depend on each other.
- **Example:**

```
C
// Compiler might fear that index[i] causes collisions
#pragma acc loop independent
for (int i = 0; i < N; i++) {
    A[index[i]] += b[i];
}
```

seq

- **Definition:** Forces the loop to run sequentially (single thread).
- **Example:**

```
C
#pragma acc parallel loop
for (int i = 0; i < N; i++) {
    // This inner loop must run in order for each 'i'
    #pragma acc loop seq
    for (int j = 1; j < M; j++) {
        row[j] = row[j-1] + val; // Dependency on previous j
    }
}
```

reduction(op:var)

- **Definition:** safely combines results from multiple threads.
- **Example:**

```
C
float total = 0.0f;
#pragma acc parallel loop reduction(+:total)
for (int i = 0; i < N; i++) {
    total += A[i];
}
// 'total' now holds the correct sum from all threads
```

1. Matrix Multiplication ($C = A \times B$)

Key Concept: Using collapse(2) to turn the 2D output grid into a massive 1D loop for better GPU saturation.

C

```
#include <stdio.h>
#include <stdlib.h>

// Flattened 2D arrays: A[N][P], B[P][M], C[N][M]
void matrixMul(float *restrict A, float *restrict B, float *restrict C, int N, int P, int M)
{
    // Data Region:
    // A and B are Input (copyin)
    // C is Output (copyout) - we don't care what was in C before
    #pragma acc data copyin(A[0:N*P], B[0:P*M]) copyout(C[0:N*M])
    {
        // Parallel Region:
        // collapse(2) merges loops 'i' and 'j' so the GPU sees N*M total threads
        #pragma acc parallel loop collapse(2)
        for (int i = 0; i < N; ++i) {
            for (int j = 0; j < M; ++j) {

                float sum = 0.0f;
```

```

        // Inner loop is a reduction (dot product).
        // We run this sequentially or vector-parallel depending on M size.
        // Usually 'seq' is safe here unless P is very large.
        #pragma acc loop seq
        for (int k = 0; k < P; ++k) {
            sum += A[i * P + k] * B[k * M + j];
        }
        C[i * M + j] = sum;
    }
}
}
}

```

2. 2D Heat Equation (Jacobi Iteration)

Key Concept: Managing data *outside* the while loop to prevent massive data transfer overhead (The "Hotspot" fix from Slide 23).

C

```

#include <math.h>
#include <stdio.h>

#define N 1024
#define MAX_ITER 1000
#define TOL 0.001f

void heat_equation(float *restrict A, float *restrict Anew)
{
    float error = 1.0f;
    int iter = 0;

    // DATA REGION: Keep A and Anew on GPU for the entire simulation
    #pragma acc data copy(A[0:N*N]), create(Anew[0:N*N])
    {
        while (error > TOL && iter < MAX_ITER) {
            error = 0.0f;

            // Kernel 1: Stencil Calculation

```

```

// async(1): Don't block CPU yet
#pragma acc parallel loop collapse(2) reduction(max:error) async(1)
for (int j = 1; j < N - 1; j++) {
    for (int i = 1; i < N - 1; i++) {
        Anew[j * N + i] = 0.25f * (A[j * N + (i + 1)] +
                                   A[j * N + (i - 1)] +
                                   A[(j - 1) * N + i] +
                                   A[(j + 1) * N + i]);

        // Calculate error on the fly
        float diff = fabsf(Anew[j * N + i] - A[j * N + i]);
        if (diff > error) error = diff;
    }
}

// Kernel 2: Copy back (Swap)
// async(1): Queued after Kernel 1
#pragma acc parallel loop collapse(2) async(1)
for (int j = 1; j < N - 1; j++) {
    for (int i = 1; i < N - 1; i++) {
        A[j * N + i] = Anew[j * N + i];
    }
}

// Wait for kernels to finish before checking 'error' on CPU
#pragma acc wait(1)

iter++;
if (iter % 100 == 0) printf("Iter %d, Error: %f\n", iter, error);
}
} // Data copied back to CPU here
}

```

3. Stencil Computations

1D Stencil (3-Point Moving Average)

Key Concept: Handling boundaries.

```

void stencil_1d(float *restrict in, float *restrict out, int n) {
    #pragma acc data copyin(in[0:n]) copyout(out[0:n])
    {
        #pragma acc parallel loop
        for (int i = 1; i < n - 1; i++) {
            out[i] = (in[i-1] + in[i] + in[i+1]) / 3.0f;
        }
        // Handle boundaries separately or inside if needed
        // (Usually handled on host or via 'if' inside kernel)
    }
}

```

General Weighted 2D Stencil

Key Concept: Applying a constant weight matrix (convolution).

C

```

#define WIDTH 1024
#define HEIGHT 1024

// Weights: 3x3 matrix
void stencil_2d_general(float *restrict in, float *restrict out, float *restrict weights) {

    // Weights are small and read-only, ideal for constant memory or copyin
    #pragma acc data copyin(in[0:WIDTH*HEIGHT], weights[0:9]) copyout(out[0:WIDTH*HEIGHT])
    {
        #pragma acc parallel loop collapse(2)
        for (int y = 1; y < HEIGHT - 1; y++) {
            for (int x = 1; x < WIDTH - 1; x++) {

                float sum = 0.0f;
                int w_idx = 0;

                // 3x3 window loop (small, run sequentially inside the thread)
                #pragma acc loop seq
                for (int wy = -1; wy <= 1; wy++) {
                    for (int wx = -1; wx <= 1; wx++) {

```

```

        sum += in[(y + wy) * WIDTH + (x + wx)] * weights[w_idx++];
    }
}
out[y * WIDTH + x] = sum;
}
}
}
}
}

```

4. Matrix Transpose

Key Concept: Memory access patterns. While collapse helps parallelism, transpose is notoriously hard on memory bandwidth (non-coalesced reads/writes).

C

```

void transpose(float *restrict in, float *restrict out, int rows, int cols) {
    #pragma acc data copyin(in[0:rows*cols]) copyout(out[0:rows*cols])
    {
        // Collapse ensures we have rows*cols threads
        // Tile(32,32) can help with cache locality (similar to Shared Memory blocking in CUDA)
        #pragma acc parallel loop tile(32, 32)
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                // out[j][i] = in[i][j]
                out[j * rows + i] = in[i * cols + j];
            }
        }
    }
}

```

5. Your Custom Code (Vector Update)

Key Concept: Multiple arrays with different data directions.

- A: Read & Write $\rightarrow$ copy

- B: Read & Write $\rightarrow$ copy
- C: Write Only $\rightarrow$ copyout (assuming we don't need previous C values)

C

```
void compute(float* restrict A, float* restrict B, float* restrict C, int N)
{
    // Define the data region explicitly to handle array sizes
    #pragma acc data copy(A[0:N], B[0:N]) copyout(C[0:N])
    {
        // Simple 1D loop parallelization
        #pragma acc parallel loop
        for(int i = 0; i < N; ++i)
        {
            A[i] = A[i] * 2.5f;
            B[i] = B[i] + 10.0f;
            C[i] = A[i] + B[i];
        }
    }
}
```

6. Bonus: Dot Product (Parallel Reduction)

Key Concept: reduction clause. This is a very common exam question type (e.g., "Calculate Pi" or "Sum of Array").

C

```
float dot_product(float *restrict A, float *restrict B, int N)
{
    float sum = 0.0f;

    #pragma acc data copyin(A[0:N], B[0:N])
    {
        // reduction(+:sum) is crucial!
```



```
// Without it, threads race to overwrite 'sum', giving garbage results.  
#pragma acc parallel loop reduction(+:sum)  
for (int i = 0; i < N; i++) {  
    sum += A[i] * B[i];  
}  
  
return sum;  
}
```